

Each task must be performed individually and with full awareness. GenAI tools can be consulted to aid comprehension, but your primary goal is to develop your own problem-solving skills through individual effort, i.e., use it to enhance your understanding of solutions, not to generate them directly.

CREATE A SHORT REPORT WITH ALL SOLUTIONS

Task 1 is part of Laboratory 0 and serves as a quick recap and verification of your SQL skills. Tasks 2-5 and all Tutorials 1-3 are part of laboratory assignment 1 and are due next week.

LAB 1

WEEK 2

CONTENT

- Final introduction to AdventureWorks database, after this lab you are on your own (you can and should use documentation)
- Relational model – design basics
- SQL reminder – querying basics
- Analytical extensions to SQL

ADDITIONAL RESOURCES

- General querying information – T-SQL querying:
 - [http://technet.microsoft.com/en-us/library/ms181080\(v=sql.105\).aspx](http://technet.microsoft.com/en-us/library/ms181080(v=sql.105).aspx)
 - <https://learn.microsoft.com/en-us/training/paths/get-started-querying-with-transact-sql/>
 - SELECT Examples (Transact-SQL) - SQL Server | Microsoft Docs:
 - <https://docs.microsoft.com/en-us/sql/t-sql/queries/select-examples-transact-sql>
 - TSQL Tutorial - Learn Transact SQL language:
 - <https://www.tsq.info>
 - Advanced T-SQL:
 - <https://learn.microsoft.com/en-us/training/modules/write-advanced-sql-code/>

TASK 1 – SQL QUERIES

SQL warmup task. Please do not use the build-in query editor – focus on pure SQL statements! Prepare adequate SQL queries to:

1. Provide information about the global sales amount (money), number of orders and volume (items sold) of the AdventureWorks business. (Use SalesOrderHeader table)
 - a. An example of the query result is shown in Table 2.1:

Table 2.1. Result structure for task 2.1

Sales Amount	Volume	Number of orders
...	...	...

2. Provide information about the sales amount, volume, and number of orders in individual years of operation of the business. (Use SalesOrderHeader table)
 - a. An example of the query result is shown in Table 2.2:

Table 2.2. Result excerpt for task 2.2

Year	Sales Amount	Volume	Number of orders
...	...	...	...

3. Prepare a SQL query that provides top 5 customers with the highest number of orders, try using the customer name (it might be tricky). (Start with SalesOrderHeader and identify needed tables)
 - a. An example of the query result is shown in Table 2.3.:

Table 2.3. Result excerpt for task 2.3

CustomerID	Last name, name	Number of orders
------------	-----------------	------------------

...	...	...
-----	-----	-----

4. Prepare a SQL query that provides the names of all individual customers with the total sum of purchases (use SalesOrderHeader.SubTotal) greater than 1500USD – sorted (descending) by the total sales amount.
 - a. An example of the query result is shown in Table 2.4.:

Table 2.4. Result excerpt for task 2.4

CustomerID	Last name, name	SalesAmount
...	...	...

5. Prepare a query that provides information about average price, total sales amount, and total volume in individual product categories of the AdventureWorks business. (Start with SalesOrderHeader and identify needed tables)
 - a. An example of the query result is shown in Table 2.5.:

Table 2.5. Result excerpt for task 2.5

CategoryID	Category name	Average price	Total Sales Amount	Total Volume
...	...	...	...	...

6. Display all subcategories which average price is higher than the average price of all categories.
 - a. An example of the query result is shown in Table 2.6.:

Table 2.6. Result excerpt for task 2.6

SubcategoryID	Subcategory Name	Average price	Average price (over all categories)
...	...	...	...

7. Select sales territory (name) with sales in May 2013 higher than the average monthly sales per sales territory. (Start with SalesOrderHeader and identify needed tables)
 - a. An example of the query result is shown in Table 2.7.:

Table 2.7. Result excerpt for task 2.7

SalesTerritoryID	Sales Territory Name	Sales (May 2013)	Average monthly sales (per territory)
...	...	...	...

8. Create a list of sales territories (ids are enough) with an average number of orders (both real value and the largest integer less than the value) made by customers who have more than 10 orders in general (use CTE)
 - a. An example of the query result is shown in Table 2.8.:

Table 2.8. Result excerpt for task 2.8

TerritoryID	Average number of orders	Average number of orders (INT)
...	...	...

9. (*) Show monthly sales amount by each sales territory in year 2013 and calculate the difference with the previous month (use 0 for 12/2012) to identify trends.
 - a. An example of the query result is shown in Table 2.9.:

Table 2.9. Result excerpt for task 2.9

TerritoryID	Sales Territory Name	Mnt Sales Amt	Diff to prev
...	...	...	...

All results should be additionally stored in a single text .sql file. After completing the task, double check your approach and please upload the file to ePortal. Note that the task allows for submission of multiple files.

TASK 2 – DATA MODELLING

Let us do a quick refresher on data modelling. We assume you are to design a relational database and you are focused on a purely operational database (OLTP). Assume that there is an initial draft of the database available, created by another individual. As such, please analyse the conceptual data model of "Orders" (Fig. 1.) and the

provided details concerning user requirements. Just note that the diagram might be incomplete, yet some of its classes and relationships between them may represent a part of the reality under consideration.

User requirements:

Consider a straightforward situation (intuitive understanding) of handling orders made by customers, for a set of products, and handled by a set of different shops. Please assume, that a customer can repeatedly shop in the same store, and any customer can shop in any store. Each purchase is made by the customer in the store on a specific day and time. For each order we need to track quantity, price and total value for each product on this order. Each customer has a default discount that is applied to the total value of its orders, yet the discount cannot be greater than 20%. Each store can individually propose the price and maintain quantity of the offered product. Each product is defined by a unique UPC code, has a name, model and associated category. Finally, the same product can be offered by multiple stores.

Proposed domain rules and constraints:

- R01 – A customer can repeatedly shop in the same store
- R02 – Any customer can shop in any store
- R03 – Each purchase is made by the customer in the store on a specific day and time
- R04 – Each store can individually propose the price and quantity of the offered product

Proposed diagram:

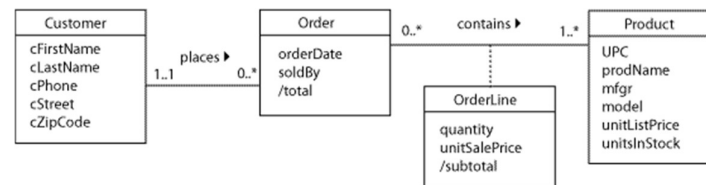


Figure 1 Conceptual data Model, UML

Then, complete the following tasks. In general, modify the set of rules and constraints (by supplementing their definition) or correct the conceptual data model (diagram, justify your changes) to obtain a valid model.

1. Specify the set of rules and domain constraints.
2. Provide a revised and complete version of the data model (complete UML class diagram)
3. Create a logical data model as a relational model. Test the created data model.

Using the above conceptual model we want to further implement it in a relation database system. Complete the following tasks:

4. Create a physical data model as a DDL SQL script (including domain rules and constraints).
 - a. Create a new database in MS SQL Server. Please note, that implemented database should be a physical data model of the modeled part of the reality and should be coherent with your conceptual and logical data model.
5. Test the database created.
 - a. Enter several records into each table, checking the correctness of the implementation (remember to check both correct data and inconsistent data with the applicable rules – please comment and explain the obtained messages from the DBMS system).

TUTORIAL – PIVOTING AND GROUPING SETS IN SQL

Let us focus on selected detailed querying capabilities of SQL. In particular, we are interested in accessing summary information from a strictly operational database. Let us assume that we want to assess the yearly performance of individual sales representatives working for AdventureWorks company. As such, we will focus on defining a set of basic SQL queries, which retrieve all the required information that can be further reported to the management.

The major goal of this task is to investigate, on a real example, how easy it is to run analytical processing tasks, here a particular query, on a strictly transactional database. Additional resources regarding needed T-SQL clauses are available below:

- Overview: <https://docs.microsoft.com/en-us/sql/t-sql/queries/queries>
- T-SQL querying: [http://technet.microsoft.com/en-us/library/ms181080\(v=sql.105\).aspx](http://technet.microsoft.com/en-us/library/ms181080(v=sql.105).aspx)
- PIVOT: <http://technet.microsoft.com/en-us/library/ms177410%28v=sql.105%29.aspx>
- CASE: <http://msdn.microsoft.com/en-us/library/ms181765.aspx>
- GROUP BY ROLL UP, CUBE and GROUPING SETS: <https://learn.microsoft.com/en-us/sql/t-sql/queries/select-group-by-transact-sql>
- OVER: <https://docs.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql> (next lab)

TASK 3 – CASE

Please prepare for each subtask – query, result (or at least its excerpt). Please use the data structures created in the previous Lab Assignments. **Use Case expression** – prepare SQL queries which as a result:

1. provides different product's price categories:
 - a. ListPrice < 20.00 – Inexpensive
 - b. 20.00 < ListPrice < 75.00 – Regular
 - c. 75 < ListPrice < 750.00 – High
 - d. 750.00 < ListPrice – Expensive
2. provides information about total volume of product for different price categories and different product categories – use price categories in columns, product categories in rows, and total volume as values (0 - never sold a product from a given category); please use CASE to put years on columns

TASK 4 – PIVOT

Pivoting – prepare SQL queries within a tool of your choice (like SQL Management Studio or Azure Data Studio). Please prepare for each subtask a query utilising **Pivot operator** which as a result:

1. provides information about product's subcategory, product's colour and total sales value; please do not use pivoting here – this will be our base query for the pivot.
2. provides information about total sales value for different colours of different product subcategories; please put colours on columns, products' subcategory names on rows, and total sales value as values; use the query from the previous point.
 - a. please prepare a version of this query, where only subcategories from category *Bike* are presented.
3. (*) provides information about average sales subtotal amounts in years and months; please put months on columns, years on rows, and subtotal as values – do this without manually specifying all individual years.

TASK 5 – GROUPING OPERATORS

Please prepare a series of SQL queries using **Grouping operators**. For each point, please prepare a single SQL query, which as a result:

1. provides total sales amount for different product categories along with a total value of sales for all categories – use only one SELECT statement (do not use UNION),
2. provides total sales amount for different products (use product's name) in different product categories and subcategories – please provide sales summaries for: each category and subcategory, each category, each subcategory and a total sales amount value (aggregated for all categories and subcategories) – use only one SELECT statement (do not use UNION),
3. provides total sales amount for each product category and colour (include products without specified colour), total for each colour, total for each category and total sales amount:
 - a. Please use grouping function to identify the total sales amount from the sales amount for products without specified colour:
 - i. <https://docs.microsoft.com/en-us/sql/t-sql/functions/grouping-transact-sql>

RESULTS

After completing the task, double check your results and please create a simple report (use code and comments). Try storing all SQL queries in a single text file (.txt/.sql). Other results should be presented and discussed in a text/Word file. After completing all the tasks, double check your results and please present your results to the teacher. Next, upload files to ePortal.

REMARKS:

- *Try to prepare a short report from your work*
- *A submission without final conclusions will not be checked and results in a negative score!*
 - *If you are not preparing a report, please use comments on Eportal to provide your conclusions.*